

MegaFold: Efficient Training of Next-Generation 3D Attention Protein Models on Cross-Platform GPUs

Hoa La^{*†}

University of Massachusetts, Amherst
hvla@umass.edu

Ahan Gupta^{*}

SSAIL Lab, University of Illinois Urbana-Champaign
ag82@illinois.edu

Alex Morehead

Lawrence Berkeley National Laboratory
acmwhb@lbl.gov

Jianlin Cheng

University of Missouri
chengji@missouri.edu

Minjia Zhang

SSAIL Lab, University of Illinois Urbana-Champaign
minjiaz@illinois.edu

Abstract—Recent advances in biomolecular modeling have been catalyzed by models such as AlphaFold3 (AF3), which introduce science-informed changes to the transformer architecture. Unlike transformers, a defining characteristic of AF3-style models is their 3D attention over 2D pairwise representations which produces tensors whose computation and memory costs scale *cubically* with sequence length. As a result, despite moderate parameter counts, AF3-style models are far more expensive to train than size-equivalent transformers, and are severely constrained by GPU memory capacity. Our characterization shows 3D attention fundamentally changes the training workload, causing massive 3D attention maps, complex inter-operator dependencies, kernel fragmentation, and heavy host-side data pipelines which differ substantially from LLM training, leading to poor utilization on modern GPU systems. Moreover, existing GPU optimizations do not adequately address these challenges due to complex cross-layer inter-operator dependencies introduced by 3D attention. Motivated by these challenges, we introduce MegaFold, a novel cross-platform system for efficient training of next-generation 3D-attention protein models. MegaFold combines a memory-efficient 3D-attention kernel, a communication-efficient sharding strategy for quadratic representations, fused operator implementations for critical execution paths, and a determinism-aware host-device pipeline that eliminates preprocessing stalls. Evaluation on both NVIDIA H200 and AMD MI250 GPUs shows that MegaFold enables training with up to $3.36\times$ longer sequence lengths on 32 GPUs while reducing end-to-end execution time by up to $1.73\times$ (NVIDIA) and $1.62\times$ (AMD).

Index Terms—High performance computing, Bioinformatics, Parallel algorithms, Performance analysis.

I. INTRODUCTION

Deep learning has transformed structural biology, enabling accurate prediction of protein structures and biomolecular interactions [1]. Models such as AlphaFold3 (AF3) extend this capability beyond single proteins to protein-protein, protein-DNA/RNA, and protein-ligand interactions [2], making a shift towards next-generation biomolecular modeling workloads [3]–[5]. Unlike earlier models, these models incorporate science-informed architectural changes that fundamentally change the computational structure of training.

A defining characteristic of AF3-style models is the use of 3D attention over 2D pairwise representations of amino acids. This design produces high-dimensional tensors whose compute and memory scale *cubically* with sequence length. As a result, despite a moderate parameter count ($\sim 500\text{M}$ parameters) [2], AF3-style models are significantly more expensive to train than size-equivalent transformer models. In practice, AF3 training can be up to two orders of magnitude slower and supports input lengths that are an order of magnitude shorter. For example, sequences longer than 590 residues lead to out-of-memory (OOM) errors even with batch size one, while a size-equivalent 560M parameter transformer such as BLOOM-560M [6] easily scales to thousands of tokens on the same GPU.

To understand this gap, we perform a detailed performance characterization of AF3 training and identify four fundamental challenges rooted in 3D attention (§ III). *Challenge-I*: AF3 training suffers from extreme memory pressure due to cubic scaling. Its core operator processes 2D pairwise representations and produces a 3D attention map with $O(N_{\text{token}}^3)$ memory complexity which easily overwhelms on-chip caches and generates memory traffic that quickly exceeds device bandwidth limits. *Challenge-II*: AF3 introduces multi-axis communication patterns that differ fundamentally from those in transformer training. To alleviate memory bottlenecks from long sequences, prior works develop sequence parallelism (SP) strategies for transformers [7], [8]. However, extending these strategies to 2D pairwise representations requires sharding along both sequence and pair dimensions. This results in 3D communication flows that are not supported by conventional parallelism schemes. *Challenge-III*: AF3 exhibits severe kernel fragmentation. To model complex biomolecular interactions, training interleaves 3D attention, transition layers, and projection operations, producing thousands of fine-grained kernels per iteration. These kernels are too small to efficiently utilize GPU execution pipelines, and lead to poor warp occupancy, underutilized memory bandwidth, and excessive intermediate activations. *Challenge-IV*: AF3 training suffers from heavy host-side preprocessing. Constructing the 2D pairwise inputs required by 3D attention involves multi-stage biological preprocessing, including multi-sequence alignment (MSA) [1], which runs on

^{*}Equal Contribution.

[†]Work done while interning at UIUC SSAIL Lab.

the CPU and introduces significant GPU idle time that cannot be hidden by conventional prefetching-based data pipelines.

Collectively, these challenges constrain AF3 training to short sequences and result in prohibitively long training times (e.g., 11 days on 128 GPUs [9] despite being a moderate $\sim 500\text{M}$ parameter model). Although modern GPU systems provide highly optimized support for transformer training, these optimizations do not transfer to AF3. Techniques such as FlashAttention [10], compiler-based fusion [11]–[13], and standard sequence parallelism [7], [14] assume 2D attention over 1D sequences and are insufficient for the cubic scaling, and cross-layer dependencies introduced by 3D attention.

Motivated by these challenges, we present MegaFold, a novel system for efficient training of next-generation 3D-attention protein models on modern GPUs. MegaFold is the first end-to-end system we are aware of that provides integrated support for 3D attention training by jointly addressing kernel execution, parallelization, operator fusion, and host-device pipelining. MegaFold introduces (1) EvoFlash-3D, a Triton-based kernel for 3D attention over 2D pairwise representations to achieve high memory and bandwidth efficiency on cross-platform GPUs (§ IV-A), (2) EvoSP-3D, a communication-efficient sharding strategy that supports alternating attention axes from 2D pairwise representations (§ IV-B), (3) EvoFusion, a fused operator stack that aligns attention and transition layers with GPU shared-memory hierarchies (§ IV-C), (4) EvoPipe, a determinism-aware host-device pipeline that eliminates CPU-side preprocessing stalls (§ IV-D).

We implement MegaFold on top of a widely used open-source AF3 training system [15] and evaluate it on both NVIDIA H200 and AMD MI250 GPUs. Our results show that MegaFold reduces peak memory usage by up to $1.23\times$, while improving per-iteration training time by up to $1.73\times$ on NVIDIA and $1.62\times$ on AMD. Moreover, MegaFold enables scalable training with up to $3.36\times$ longer input sequence lengths on 32 GPUs. These results demonstrate that 3D-attention protein models introduce novel system-level challenges, and that dedicated system support is essential for scaling emerging biomolecular workloads on modern GPU platforms.

II. BACKGROUND

In this section, we provide the necessary background on AlphaFold3 (AF3) [2], focusing on architectural features that give rise to its unique system challenges. AF3 [2] extends AF2 [1] from single-protein folding to modeling interactions among proteins, DNA, RNA, and small molecules. To support this generality, AF3 introduces a substantially more complex architecture and data pipeline than transformer-based language models. As shown in Fig. 1, AF3 consists of three stages: data loading and preprocessing, the Pairformer backbone, and a diffusion-based structure decoder.

Inputs and representations. AF3 processes raw biomolecular inputs into two primary internal representations. The *single representation*, denoted as token with shape $[\text{N}_{\text{token}}, \text{h}_{\text{single}}]$, which captures per-token features such as chemical identity and local geometry. To model interactions between tokens, AF3

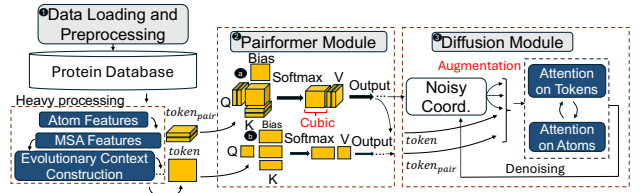


Fig. 1: Overview of the AlphaFold3 (AF3). AF3 consists of 3 stages: (1) data loading and preprocessing, which constructs single-token and pairwise representations from input biomolecular complexes using evolutionary context; (2) the Pairformer backbone, which jointly processes 1D token and 2D pairwise representations using 3D attention with cubic complexity; and (3) a diffusion-based decoder, which generates 3D atomic coordinates through iterative denoising with data augmentation.

also constructs a *pair representation*, $\text{token}_{\text{pair}}$, with shape $[\text{N}_{\text{token}}, \text{N}_{\text{token}}, \text{h}_{\text{pair}}]$. This quadratic representation is initialized from geometric priors and enhanced using evolutionary context from multiple sequence alignment (MSA) and structural information from template complexes. These representations become the core inputs to the Pairformer module.

Pairformer and 3D attention. The Pairformer module is one of the main computational backbones of AF3 and replaces Evoformer used in AF2. It operates jointly on the 1D token representation and the 2D $\text{token}_{\text{pair}}$ representation using two primary operators: EvoAttention, and transition layers. EvoAttention is a 3D attention operator that computes attention over the 2D pair representation, while conditioning the computation on sequence-level features derived from the 1D token representation via broadcasted projections and bias terms. This mixed-dimensional attention produces intermediate tensors whose size scales cubically with sequence length, introducing substantial activation memory and complex data dependencies that differ from standard self-attention over 1D textual sequences. The transition layers further process pairwise features using layer normalization and feed-forward networks with SwiGLU [16] activations, which preserves these mixed-dimensional dependencies across layers.

Diffusion-based structure decoding. Following Pairformer, AF3 employs a diffusion-based decoder to generate 3D atomic coordinates conditioned on the learned representations. The decoder applies attention over atomic and token representations and iteratively denoises predicted structures. To improve robustness and learning signal, AF3 performs extensive data augmentation, which augments 3D atomic coordinates (e.g., by $48\times$) with random rotations and translations, increasing the effective batch size processed by the diffusion module.

III. PERFORMANCE CHARACTERIZATION

We characterize four performance challenges by analyzing a widely used open-source AlphaFold3 implementation [15].

A. 3D Attention Induces Cubic Activation Memory Growth

AF3 has a moderate parameter count ($\sim 500\text{M}$), yet incurs significant activation memory during training. This limits the maximum sequence length that can be supported and

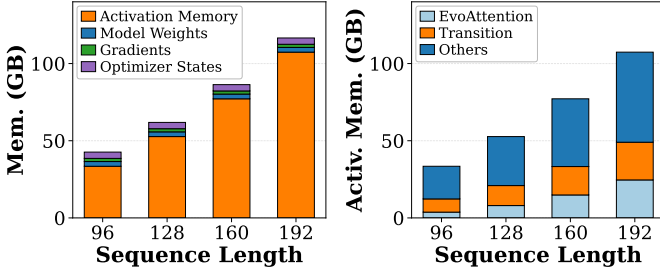


Fig. 2: End-to-end memory breakdown of AF3 training across sequence lengths. Activation memory dominates memory consumption and grows rapidly with sequence length, while model parameters, gradients, and optimizer states remain relatively stable.

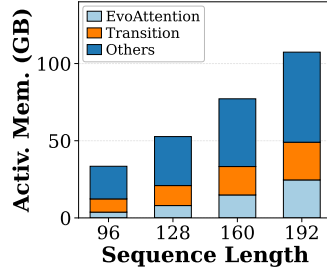


Fig. 3: Breakdown of activation memory across operator groups in AF3. EvoAttention exhibits fastest growth with sequence length due to its cubic tensors, while activations from length transition and other operators increases and remains a substantial fraction of the total.

forces models to train relatively small biomolecular complexes. To quantify this effect, we measure AF3’s end-to-end memory consumption across a range of sequence lengths in Fig. 2, breaking down memory consumption into model weights, activation, gradients, and optimizer states. We observe that while model weights, gradients, and optimizer states collectively consume only 4.08GB of memory, activation memory dominates overall usage, reaching up to 107.4GB, 96% of total memory consumption. Fig. 3 further shows the breakdown of the activation memory by operator. Two operators, EvoAttention and Transition, are responsible for nearly $\sim 50\%$ of the total activation footprint, as they frequently appear across both the Pairformer and Diffusion modules. Among these, EvoAttention is the primary contributor to memory growth with increasing sequence length. At a sequence length of 96, EvoAttention produces ~ 3.75 GB of activations, which increases to ~ 24.61 GB at a sequence length of 192. This corresponds to a $6.56\times$ increase in activation memory for a $2\times$ increase in sequence length, reflecting the cubic complexity of 3D attention over 2D pairwise representations. As sequence length increases, the resulting activation tensors overwhelm on-chip caches and exceed GPU HBM capacity, leading to OOMs while also generating memory traffic that saturates device bandwidth even when execution is feasible.

Insight-1: 3D attention over 2D pairwise representations generates activations whose size grows cubically with sequence length. These activations dominate GPU memory usage, exceed HBM capacity at moderate sequence lengths, and stress cache locality and device bandwidth.

B. 2D Pairwise Representations Break Sequence Parallelism

AF3 models are typically trained at scale with distributed data parallelism (DDP), largely because their moderate parameter count fits within the memory of a single GPU. While DDP effectively scales batch size, it does not scale input sequence length. We confirm this behavior by scaling AF3 training from 2 to 8 GPUs while holding the per-GPU micro-batch size fixed at

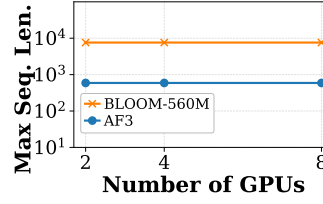


Fig. 4: Max Seq. length trainable for AF3 and BLOOM-560M under DDP as GPU count increases. Despite similar model sizes, AF3 supports substantially shorter sequences due to cubic activation in 3D attention.

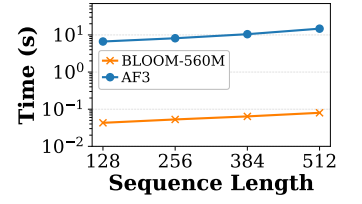


Fig. 5: Average training iteration time of AF3 and BLOOM-560M on a single GPU across sequence lengths. AF3 exhibits significantly higher per-iteration cost compared to the transformer-based BLOOM-560M.

1 (Fig. 4). Despite the additional GPU memory, training beyond a single sequence length of 590 residues consistently results in OOMs, indicating that conventional data-parallel scaling cannot translate increased hardware resources into longer sequence support for AF3. A natural next step would be to apply sequence parallelism (SP) [7], [14], which has been shown to scale sequence length effectively for transformer models. However, extending SP to AF3 is substantially more complex due to its use of 2D pairwise representations $\text{token}_{\text{pair}}$ ($[\text{N}_{\text{token}}, \text{N}_{\text{token}}, \text{h}_{\text{pair}}]$ -sized) and attention operators that alternate their reduction axes across layers, which introduces communication and synchronization requirements that differ from those in 1D sequence models. As a result, supporting such an execution pattern requires sharding mechanisms that go beyond conventional SP designs developed for transformers. We describe such a design in § IV-B.

Insight-2: AF3 makes SP scaling substantially more complex than in transformers due to its use of 2D pairwise representations and attention operators with alternating reduction axes. As a result, extending SP to AF3 requires sharding and coordinating multi-dimensional tensors in ways that differs from the 1D SP designs commonly used for transformers.

C. Mixed-Dimensional Operators Prevent Effective Kernel Fusion

Despite AF3’s moderate model size, training invokes a large number of fine-grained GPU kernels per iteration. This stems from the model’s extensive use of mixed-dimensional operators that interleave computations over 1D sequence features, 2D pairwise representations, and higher-order intermediates. To quantify the resulting execution behavior, we profile kernel launches in a single AF3 training iteration and compare them against a size-equivalent transformer-based transformer model, BLOOM-560M [6], as shown in Fig. 6. We observe that AF3 launches approximately 36,000 linear-layer kernels, 23,000 layer-normalization kernels, and 11,000 activation-function kernels, over two orders of magnitude more than BLOOM-560M. These kernels are small and often memory-bound, which reflect the fine-grained structure induced by repeatedly transitioning between operators defined over different tensor

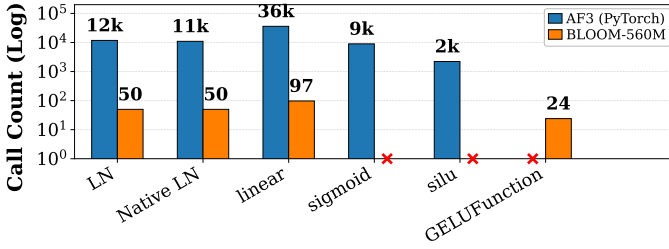


Fig. 6: Kernel launch counts (log scale) for common operators in a single training iteration of AF3 and BLOOM-560M. AF3 invokes orders of magnitude more small kernels than the size-equivalent transformer models.

dimensionalities. As a result, AF3’s execution graph is highly fragmented, preventing kernels from amortizing launch overhead or efficiently utilizing GPU execution pipelines.

The fragmentation has direct consequences to both runtime and memory efficiency. Frequent kernel launches incur accumulated scheduling and dispatch overhead, while the inputs to *each* kernel must be retained for gradient computation during the backward pass, increasing activation memory pressure. Fig. 5 shows the average per-iteration time of AF3 and BLOOM-560M across different sequence lengths. Across all configurations, AF3’s iteration time is consistently two orders of magnitude higher than that of the size-equivalent transformer. The root cause of this behavior lies in the structure of AF3’s operators. AF3 interleaves 3D attention, transition layers, triangular updates, and normalization operations that operate over different dimensional representations and exhibit non-trivial reduction semantics. As a result, reducing this fragmentation, whether through fusion or other optimizations, needs to coordinate reductions, broadcasts, and data layouts across operators with incompatible dimensional assumptions.

Insight-3: AF3’s extreme kernel fragmentation is a structural consequence of its mixed-dimensional execution. Training interleaves operators over 1D sequence features, 2D pairwise representations, and higher-order intermediates, creating fine-grained kernels with incompatible reduction and data layout semantics. These cross-operator dependencies prevent kernels from being easily fused together, introducing significant kernel launch and activation overhead.

D. Multi-Representation Evolutionary Context Construction Slows the Data Pipeline

Although AF3 operates on relatively short biomolecular complex lengths due to its large activation memory footprint, host-side data loading and preprocessing consume a substantial fraction of end-to-end training time. This overhead primarily comes from the construction of evolutionary context required to generate 2D pairwise representations used by EvoAttention. We profile 50 iterations of an AF3 training loop and decompose each iteration into data-loading and preprocessing (DLP), forward pass, backward pass, and optimizer step, as shown in Fig. 7. On average, data-loading and preprocessing consumes 38.2% of total iteration time, excluding a single outlier caused by unusually long MSA ($\sim 16k$ residues). Moreover, there is a

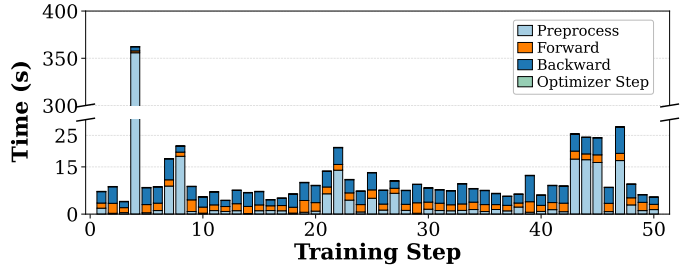


Fig. 7: Breakdown of AF3 training iterations into data-loading and preprocessing, forward pass, backward pass, and optimizer step. Data loading and preprocessing exhibits variable overhead across iterations, leading to frequent GPU idle time.

large variance in runtime across iterations due to differences in input sequence lengths and evolutionary context, leading to frequent GPU idle periods.

The GPU idling stems from the complex dependencies and I/O-bound preprocessing. AF3’s DLP stage prepares multiple representations: pairwise representations for EvoAttention processing, alongside single, MSA, template, atomic and tokenized representations for processing by other operators. Certain representations have a dependency on the creation of another (e.g. tokenized representations are aggregated through atomic representations) with I/O-bound retrieval to gather similar templates of the input complex (to enrich evolutionary contexts) dominating runtime. These sequential dependencies and I/O-bound retrieval steps make applying data-prefetching optimizations to mitigate host-side delays challenging. Increasing the number of data loading process only marginally reduces the per-iteration time.

Insight-4: Constructing evolutionary context for multiple representations introduces dependency-heavy and IO-bound preprocessing, causing persistent GPU idle time that cannot be easily eliminated by conventional data-prefetching.

IV. MEGAFOLD DESIGN

To address the bottlenecks identified in § III, MegaFold adopts a design that directly targets the sources of inefficiency in AF3 training. Fig. 8 shows an overview of MegaFold: (1) EvoFlash-3D addresses the cubic activation memory and bandwidth pressure introduced by 3D attention § IV-A, (2) EvoSP-3D enables longer sequence training over 2D pairwise representations by restructuring parallelism § IV-B, (3) EvoFusion reduces kernel fragmentation caused by mixed-dimensional operators § IV-C, and (4) EvoPipe mitigates host-side stalls introduced by evolutionary context construction § IV-D. Together, these components form a cross-stack system that enables efficient AF3 training on modern GPU platforms.

A. EvoFlash-3D: Cross-Platform Implementation of Memory-Efficient 3D Attention

AF3’s EvoAttention operator is the dominant source of activation memory pressure during training. As shown in § III-A, EvoAttention operates on 2D pairwise representations and produces 3D attention map that scales cubically with sequence

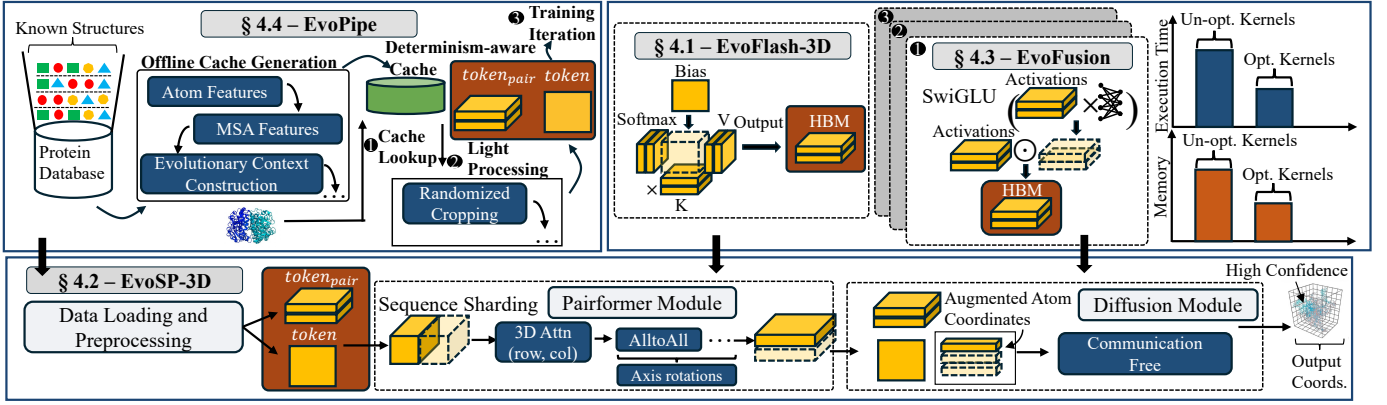


Fig. 8: Overview of MegaFold and how its optimizations are coordinated across the training pipeline. EvoPipe leverages determinism-aware cache mechanism to produce token and $\text{token}_{\text{pair}}$ representations. These representations flow through the Pairformer and diffusion stages, where EvoFlash-3D executes memory-efficient 3D attention, EvoSP-3D shards pairwise representations and coordinates inter-device communication, and EvoFusion reduces kernel fragmentation in transition layers across stages. Together, these optimizations enable efficient long-sequence AF3 training.

length. Supporting long-sequence training therefore requires a kernel design that avoids materializing these intermediate tensors while preserving the correctness of EvoAttention in both forward and backward gradient computation.

Designing such a memory-efficient kernel is non-trivial. EvoAttention normalizes attention logits using softmax before computing the output, which conventionally requires materializing the full attention matrix. Prior work such as FlashAttention [17], addresses this issue by algorithmically restructuring attention calculation using online, numerically stable softmax [18]. However, EvoAttention differs fundamentally in that it operates over 2D pairwise inputs and introduces an additive broadcasted pair-bias term. This results in additional broadcast operations in the forward pass and extra reduction operations in the backward pass, introducing multi-dimensional data dependencies that requires a specialized kernel design.

We introduce EvoFlash-3D, a memory-efficient kernel for EvoAttention that avoids materializing the cubic attention logits in global memory. Instead, EvoFlash-3D incrementally computes attention logits within fast scratchpad memory during both forward and backward passes. This design asymptotically reduces the peak activation memory of EvoAttention from $O(N_{\text{token}}^3)$ to $O(N_{\text{token}}^2)$ while maintaining numerical correctness. EvoFlash-3D is implemented using Triton [19] and is therefore performance-portable.

Fig. 9 illustrates the forward pass of the kernel, demonstrating a thread-block’s access patterns over the Q , K & V tensors as well as the rows of the attention logits a block computes. It launches 2D thread blocks: $[b_y, b_x]$, with each block computing one “chunk” (consisting of multiple rows) of the output tensor o . Because EvoAttention requires normalizing attention logits prior to being multiplied by the values, EvoFlash-3D adopts an online softmax strategy [18]. Listing 1 shows the forward pass as `fused_attn_fwd`. Each thread block maintains lightweight metadata, which consists of a running normalization factor li and a running maximum mi , both initialized locally as 0 and $-\text{inf}$ respectively (line

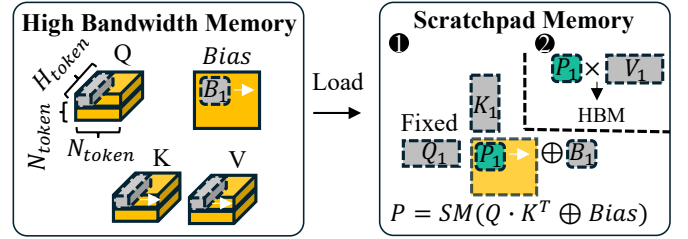


Fig. 9: EvoFlash-3D’s forward pass on a 2D pairwise input. Green boxes represent computations performed within a thread block using scratchpad memory, while gray boxes indicate accesses to queries, keys, values & bias stored in HBM. The attention-map, P , is accumulated on the fly and never materialized in HBM, reducing peak memory of EvoAttention.

9). During execution, the block first loads its assigned query vectors Q (line 10), then iterates over tiles of the key and value tensors. In each iteration, the kernel computes the query-key products and applies the corresponding pairwise bias via on-the-fly broadcasting (line 13-14). The attention logits are shifted by the current running maximum (line 16), after which the metadata (li, mi) are updated to maintain numerical stability (line 20). Partial outputs oi are normalized using the mi - running sum - metadata (line 21-22), without ever materializing the full attention matrix. After all key-value tiles are processed, the accumulated outputs are normalized using the final metadata values. The resulting log-sum-exp terms are stored to enable correct re-normalization during the backward pass.

The backward computation is decomposed into two stages to accommodate different parallel strategies required to compute the gradient tensors. One stage computes the gradients with respect to the value and key tensors (dV & dK), while the other computes gradients for the queries and pairwise bias terms (dQ & $dBias$). Notably, $dBias$ gradients require reductions across attention dimensions. This is achieved by repeatedly accumulating local partial gradients into the corresponding global location via `atomic_adds`. This strategy avoids

```

1 # Omit indexing on batch/head/hid. dim for brevity
2 def fused_attn_fwd(Q,K,V,mask,bias):
3     """Launched with thread-blocks of size: [by, bx]
4     Q, K, V: [b, heads, N, N, d] tensor.
5     mask: [b, N, N] tensor.
6     bias: [b, heads, N, N] tensor.
7     """
8     i = tl.program_id(0)
9     li, mi = [0]*by, [-inf]*by
10    qi = Q[i*by:(i+1)*by, :]
11    oi = zeros((by, d))
12    for j in range(N/bx):
13        s = tl.matmul(qi, k[j*bx:(j+1)bx, :].T) + \
14            bias[i*by:(i+1)by, j*bx:(j+1)bx]
15        mij = tl.maximum(mi, tl.max(s, 1))
16        s -= mij
17        p = tl.exp(s)
18        lij = tl.sum(s, axis=-1)
19        alpha = tl.exp(mi-mij)
20        li, mi = li*alpha+lij, mij
21        oi = oi * alpha + \
22            tl.matmul(p, v[j*bx:(j+1)bx, :])
23    return oi, li, mi

```

Listing 1: Pseudocode for the memory-efficient fused EvoAttention kernel using EvoFlash-3D. The kernel computes 3D attention over 2D pairwise inputs using stable online softmax, accumulating attention outputs on the fly and avoiding materialization of the cubic attention tensor in global memory.

retaining partially reduced dBias gradients in scratchpad memory and preserves kernel occupancy.

B. EvoSP-3D: Communication-Efficient Sharding For 2D Pairwise Representations

As shown in § III, SP is not directly applicable to AF3 training due to the use of mixed-dimensional representations and inter-operator dependencies. We therefore design EvoSP-3D, a parallelization strategy tailored to 2D pairwise representations, which enables scalable training without repeated resharding or synchronization between operators.

In EvoSP-3D, DP groups operate on different inputs within the batch, while the notion of SP groups is reused as a coordination mechanism to partition and rearrange 2D pairwise representations across stages of AF3’s execution pipeline. Multiple SP groups form a DP group, enabling batch-size scaling while allowing stage-specific partitioning tailored to AF3’s mixed-dimensional operators. We illustrate the execution of EvoSP-3D using a 2x2 (DPxSP) configuration in Fig. 10. During data loading and preprocessing, two tensors are produced, a sequence-level representation token with shape $[N_{\text{token}}, h_{\text{single}}]$, and a pairwise representation $\text{token}_{\text{pair}}$ with shape $[N_{\text{token}}, N_{\text{token}}, h_{\text{single}}]$. Because token is relatively small, it is replicated within each DP group prior to Pairformer execution. In contrast, $\text{token}_{\text{pair}}$ is initially partitioned along one sequence axis, with different SP groups operating on disjoint contiguous shards. This is communication-free, as each group drops a different portion of the sequence.

Within the Pairformer module, execution is organized as a sequence of pipeline stages operating on the 2D $\text{token}_{\text{pair}}$ representations. Each stage applies EvoAttention with a fixed sharding axis of the $\text{token}_{\text{pair}}$ tensor. Across successive stages, EvoAttention alternates the semantic axis along which attention is computed, requiring the sharding axis to rotate accordingly. To transition between stages with different shard-

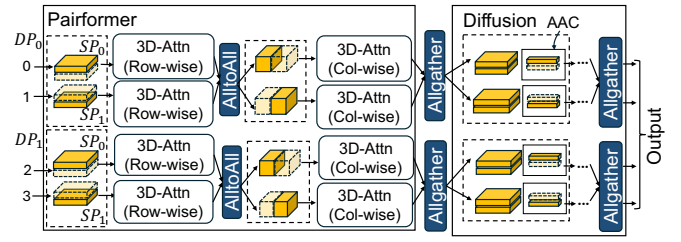


Fig. 10: EvoSP-3D with two DP and SP groups. Within Pairformer, successive EvoAttention operate on different axes of the 2D $\text{token}_{\text{pair}}$ representation. EvoSP-3D uses all_to_all to rotate the sharding layout at 3D attention boundaries so that each 3D attention receives the required 2D slices. In the diffusion stage, augmented atom coordinates (AAC) are distributed across SP groups without communication.

ing axes, all_to_all collectives are used to rearrange the layout of $\text{token}_{\text{pair}}$. Each all_to_all exchanges data proportional to the locally held shard, resulting in an aggregate communication volume proportional to $O(N_{\text{token}}^2 h_{\text{token}})$. These collectives transform the layout of $\text{token}_{\text{pair}}$ from one stage into the layout required by the next, enabling successive EvoAttention stages to operate on the correct 2D slices without materializing the full quadratic tensor. In addition, EvoAttention’s additive bias term, derived from $\text{token}_{\text{pair}}$, is replicated across device via an all_gather that is launched asynchronously and overlapped with computation, avoiding additional critical path overhead. After Pairformer processing, the updated $\text{token}_{\text{pair}}$ tensor is passed to the diffusion module together with a noised atom-coordinate tensor.

After Pairformer processing, the updated $\text{token}_{\text{pair}}$ tensor is passed to the diffusion module together with augmented atom coordinates. As described in § II, diffusion module applies random rotations and translations to generate multiple noised atom coordinates per input. EvoSP-3D exploits this structure by distributing augmented atom coordinates across SP groups, while $\text{token}_{\text{pair}}$ is all_gathered so that diffusion operates on the full pairwise representation. During the forward pass, each SP group processes its local augmented atom coordinates independently without communication. After diffusion, denoised atom coordinates are all_gathered on all SP groups to reconstruct the full output.

By allowing partitioning and communication patterns to vary across stages, EvoSP-3D avoids repeated reshaping and synchronization between incompatible representations, which would otherwise arise from naively applying SP. This design enables scalable execution over 2D pairwise representations while preserving batch-level scaling through DP, making it well suited to AF3’s training execution patterns.

C. EvoFusion: Fused Operator Stack

As shown in § III-C, AF3 training suffers from kernel fragmentation, particularly in transition layers that interleave normalization, linear projection, and activation operators. These operators are individually lightweight, but are executed as thousands of fine-grained kernels per iteration, leading to

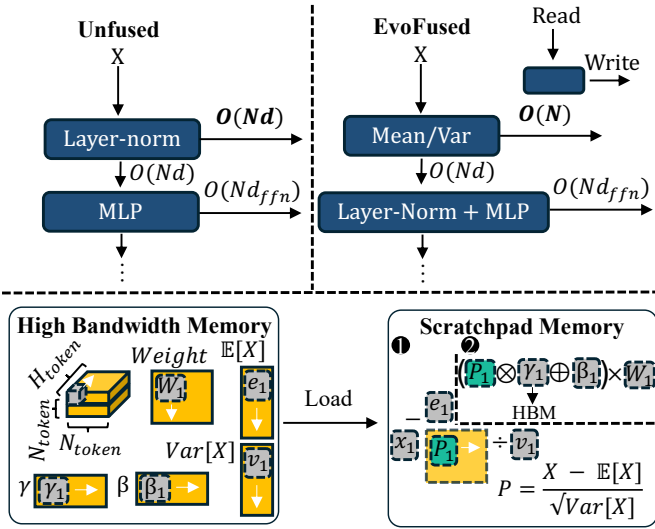


Fig. 11: EvoFusion’s forward pass for layer-norm & linear-layer fusion compared to unfused counterpart. Gray and green blocks represent values read and computed by a thread-block, respectively. By modifying inner-matmul loops, EvoFusion reduces the data written to memory.

substantial kernel launch overhead, intermediate activation materialization, and excessive memory traffic.

A key challenge is that AF3’s transition layers operate over mixed-dimensional representations and involve distinct computation patterns. Specially, layer normalization performs reductions over the hidden dimension, linear layers perform dense matrix multiplication over pairwise tensors, and activation functions apply element-wise transformations. These operators therefore have different iteration spaces, data access patterns, and synchronization requirements. Although epilogue/prologue fusion [13] and schedule-rewrite based techniques [11] can compose such operators at their boundaries, doing so forces them into a single execution schedule that is ill-suited for at least one component. In practice, this often degrades performance by increasing register and shared-memory pressure, reducing occupancy, or introducing inefficient reduction strategies, offsetting the benefits or reduced kernel launches. Consequently, addressing kernel fragmentation in AF3 requires fusion strategies that go beyond composing operators at their boundaries. Effective optimization must reorganize operator internals to preserve efficient reduction and GeMM schedules while eliminating redundant intermediate materialization.

To address these issues, we introduce EvoFusion, a fused operator stack that targets the most frequent and performance-critical operators in AF3. EvoFusion provides fused Triton kernels for layer normalizations (layer-norms), linear projections, and SwiGLU activations, which collectively account for a large fraction ($\sim 20\%$) of activation memory and kernel invocation during training. Rather than composing operators at their boundaries, EvoFlash-3D restructures their internal execution to reduce kernel launches and intermediate tensor materialization while preserving efficient computation schedules.

Fused layer-norm & linear projection. The layer-norm and

linear project fusion implementation follows a two-stage design, as shown in Fig. 11. In the forward pass, the first kernel computes per-row mean and variance of the input token tensor. A thread-block operates on an entire row in token, with M thread-blocks launched (M being the number of rows in token). A second kernel then performs normalization and matrix multiplication to compute the final output, o_2 , with each thread-block computing a single $[b_m, b_n]$ -sized tile of o_2 . Each thread-block proceeds by first zero-initializing a $[b_m, b_n]$ -sized tile, where its local answer of o_2 will be computed. Then, an inner-loop over the hidden dimension first loads a chunk of the input token tensor - c_t - layer-norm data (mean, variance of input & γ, β parameters) - c_E, c_{Var} & c_γ, c_β - and linear-layer weights - c_w . Using these values to first normalize the input chunk, computing: $((c_t - c_E)/c_{Var}) \otimes c_\gamma + c_\beta$, and then finally multiplying these normalized values with the linear-layer weights. Structuring the forward pass in this manner, as opposed to computing everything in a single kernel, reduces shared-memory pressure as fewer intermediate values need to be stored, and results in good occupancy. Moreover, it avoids storing a $token_{pair}$ sized tensor to global memory, achieving the best of both worlds: reducing memory while increasing speed. The backward pass follows a similar structure.

Fused SwiGLU. The SwiGLU activation applies an element-wise gating function of $token_{pair} \oplus \text{sigmoid}(token_{pair}) \oplus X$, where $X = token_{pair} W_1$ and W_1 are the weights of a linear-layer. EvoFusion implements SwiGLU as a single fused kernel for the forward pass, and a single kernel for the backward pass, eliminating intermediate activations and reducing kernel launch overhead while preserving per-row parallelism.

Fused transition layer. Together, the fused layer-norm-linear kernel, the fused SwiGLU activation, and the subsequent linear projection form a restructured transition layer. By eliminating materialization boundaries across these operators, EvoFlash-3D reduces kernel fragmentation and activation memory while preserving efficient execution schedules for training.

D. EvoPipe: Determinism-Aware Host-Device Pipeline

As shown in § III-D, AF3 training incurs substantial host-side overhead due to dependency-heavy and IO-bound preprocessing for constructing evolutionary contexts, leading to frequent GPU idle time. This overhead arises from science-specific preprocessing steps such as RDKit-based atom construction, the generation of MSA, and template processing, which are executed on the CPU and sit on the critical path of training. EvoPipe is designed to mitigate this bottleneck by decoupling deterministic preprocessing from the per-iteration training loop.

An AF3 training iteration consists of three stages. In the first stage, the input biomolecular complex is analyzed to construct atomic-level representations, such as atom shapes and coordinates, and to parse and tokenize MSA files to extract evolutionary features. These operations rely on expensive RDKit routines and often process long MSAs, making this stage both compute-intensive and highly variable in runtime. In the second stage, atomic and MSA features undergo additional preprocessing, such as random cropping to a target sequence

length and tokenization. Finally, in the third stage, the resulting tokenized representations are consumed by the model to perform a forward and backward pass.

EvoPipe exploits the observation that a subset of operations in the first stage produce deterministic outputs for a given input complex. In particular, RDKit-based atom construction and the conversion of raw MSA files into feature tensors do not depend on stochastic augmentation. EvoPipe precomputes these deterministic features once prior to training and stores them in a cache that maps each input complex to its corresponding atomic and MSA representations. During training, these cached features are retrieved through lightweight lookups and aligned with the current input sequence, which eliminates repeated execution of expensive preprocessing steps.

Non-deterministic operations, such as random cropping and data augmentation, remain in the per-iteration training loop and are applied on top of the cached features. By separating deterministic preprocessing from iteration-time execution, EvoPipe significantly reduces host-side overhead and variability, enabling more consistent GPU utilization during AF3 training.

V. EVALUATION

We evaluate MegaFold to answer the following questions: (1) does it significantly improve the trainability and performance of AF3-style models, (2) does it scale efficiently across multiple GPUs, and (3) how much does each system optimization contribute to the end-to-end gains. We report end-to-end training performance on both NVIDIA and AMD GPUs.

A. Evaluation Methodology

Hardware Testbed. We evaluate MegaFold on two GPU platforms: NVIDIA H200-141GB and AMD MI250-64GB GPUs. Each NVIDIA node contains $8 \times$ H200 GPUs connected via NVLink, while each AMD node contains $4 \times$ MI250 GPUs.

Datasets. We use the same data sources and preprocessing pipeline as AF3-style models, including experimentally determined structures from Protein Data Bank (PDB) [20], together with MSA and template information required for EvoAttention. The dataset covers a diverse set of protein and biomolecular complexes with varying sequence lengths.

Implementation. We implement MegaFold on top of a widely used open-source framework AlphaFold3-PyTorch [15]. We use PyTorch 2.7.0, Triton 3.2.0 with CUDA 11.8 on NVIDIA and ROCm 6.3 on AMD GPUs. EvoPipe has a bounded host-side cache for evolutionary context construction. In our experiments, this cache occupies 395GB disk space for the PDB dataset and is built offline in approximately 48 hours using 16 CPU for the entire AF3 training dataset (127,234 input complexes). Cache construction is embarrassingly parallel across input complexes and can be significantly accelerated by using additional CPUs. Moreover, this preprocessing step is amortized across training iterations and does not lie on the critical path of GPU execution. We observe no measurable impact on end-to-end scalability from EvoPipe cache management.

Baselines. We compare MegaFold against the default AlphaFold3-PyTorch training pipeline [15] executed in PyTorch

Eager Mode, which represents the reference implementation used in practice. On NVIDIA GPUs, we additionally compare against DeepSpeed-Evoformer [21], which provides hand-optimized CUTLASS kernels specialized for AlphaFold-style workloads. For completeness, we also evaluate PyTorch Inductor [13] as a representative compiler baseline.

Training Configuration. Unless otherwise stated, all experiments use a batch size of 1 with activation checkpointing (AC) [22] for both MegaFold and baselines. This configuration follows prior AF3 training practice and is needed to support practical sequence lengths under GPU memory constraints. We conduct all experiments on AF3 using the same optimizer and hyperparameters as described in AF3 [2]. For multi-GPU experiments, we additionally use distributed data parallelism (DDP) [23]. For MegaFold, we set EvoSP-3D with a DP group size of 2, and SP group sizes that scale with GPU count (e.g., 2, 4, 8, 16, 32) unless otherwise noted. Baseline systems use pure DP with the DP group size equal to the total number of GPUs.

Metrics. We evaluate MegaFold using metrics that capture end-to-end performance, memory efficiency, and scalability. These include per-iteration training time, TFLOPS, peak GPU memory usage, strong/weak scaling efficiency and Model Flops Utilization (MFU). MFU measures the efficiency of an AI training workload, quantitatively measuring the fraction of throughput a workload achieves over the hardware’s peak, accounting for only useful work (removing FLOPs due to AC).

B. Main Results

We evaluate MegaFold’s end-to-end training performance on AF3-style models, focusing on three key metrics: (1) maximum trainable sequence length, which determines the size of biomolecular complexes that can be modeled during training, (2) per-iteration execution time, and (3) peak GPU memory consumption. We compare MegaFold against PyTorch Eager (PyTorch) and PyTorch Inductor (Inductor) baselines on both NVIDIA H200 and AMD MI250 GPUs.

1) Maximum Trainable Sequence Length: Fig. 12 shows the maximum trainable sequence length under different GPU counts. MegaFold substantially improves max sequence trainability, enabling up to $3.36 \times$ longer input sequences compared to PyTorch baselines on 32 AMD GPUs. While baseline systems encounter OOM failures at moderate sequence lengths due to cubic activation growth in 3D attention, MegaFold consistently supports longer sequences. DeepSpeed-Evoformer (DS-Evoformer) relies on hand-written CUTLASS kernels and is therefore only evaluated on NVIDIA GPUs, as it is not compatible with AMD platforms. The gains in maximum trainable sequence length come from two complementary effects. On a single GPU, EvoFlash-3D and EvoFusion reduce the activation footprint, enabling up to 35% longer sequence (Fig. 19, left). In multi-GPU settings, EvoSP-3D further extends this boundary by sharding quadratic pairwise representations across devices, enabling another up to $3 \times$ longer sequence. Together, kernel-level memory optimizations and distributed sharding substantially shift the OOM barrier.

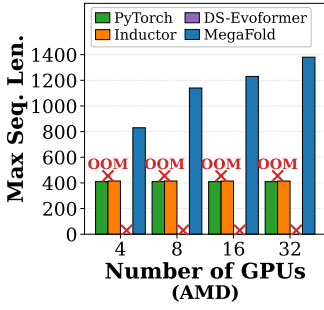


Fig. 12: Maximum trainable sequence length before OOMs for different systems as GPU count increases. MegaFold scales to substantially longer sequences than PyTorch baselines, while DS-Evoformer is unsupported on AMD.

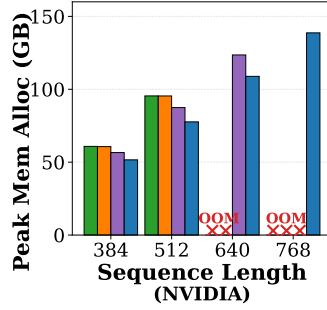


Fig. 13: Per-device peak memory consumption prior to OOMs across sequence lengths on NVIDIA GPUs. MegaFold reduces peak memory usage compared to PyTorch baselines, enabling training at longer sequence lengths.

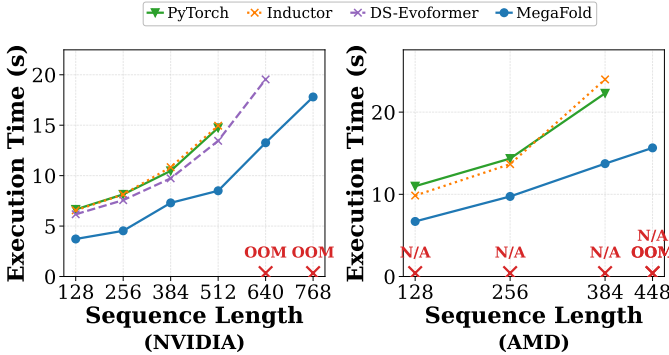


Fig. 14: Average per-iteration execution time of AF3 training on a single GPU across sequence lengths. Results are shown on NVIDIA (left) and AMD (right) hardware. MegaFold achieves speedups and supports longer sequences compared to baselines.

2) *End-to-End Speed*: Fig. 14 reports the comparison of per-iteration training time across a range of sequence lengths. MegaFold reduces iteration time by up to $1.73\times$ on NVIDIA H200 and $1.62\times$ on AMD MI250 compared to PyTorch Eager, while simultaneously supporting longer sequences. In contrast, PyTorch Inductor provides little benefit for AF3 workloads due to graph breaks introduced by mixed-dimensional operators, which limit cross-operator fusion. Performance gains grow with sequence length, which reflects the increasing effectiveness of MegaFold’s memory-efficient kernels and fused operators under high activation pressure.

3) *Memory Efficiency*: Fig. 13 shows peak GPU memory usage during training. MegaFold reduces peak memory consumption by up to 23% relative to the PyTorch baselines, allowing memory savings to be reallocated toward longer sequences. As a result, MegaFold is able to train up to sequence length 768, whereas baselines run OOMs. These reductions stem from avoiding materialization of cubic intermediate tensors in 3D attention and eliminating redundant intermediate activations in transition layers.

The evaluation shows that MegaFold’s performance gains

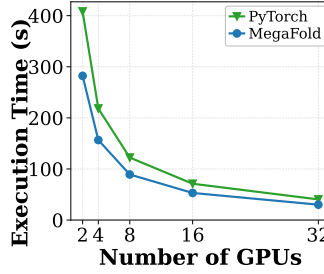


Fig. 15: Strong Scaling of AF3 with a fixed batch size. As GPU count increases, execution time differences narrow as communication overheads increasingly dominate.

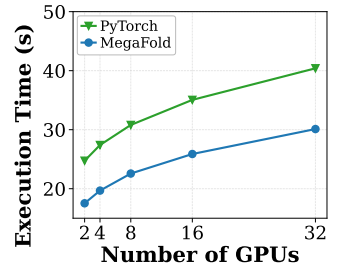


Fig. 16: Weak Scaling of AF3 with batch size proportional to GPU count. MegaFold maintains lower execution time than the PyTorch baselines as GPU count increases.

align with its system design. Memory-efficient 3D attention EvoFlash-3D and fused transition operators via EvoFusion shift the OOM barrier to support longer sequences, while EvoSP-3D and EvoPipe improve training scalability and speed. Together, these techniques enable efficient, cross-platform training of AF3-style models.

C. Scalability Evaluation

We evaluate MegaFold’s scalability to assess whether its communication and pipeline design remains efficient as GPU count increases. We report both strong and weak scaling results on AMD MI250 GPUs, comparing MegaFold against PyTorch Eager Mode, which provides the strongest baseline on this platform. All experiments use a fixed sequence length of 384 with EvoSP-3D set to 2.

Strong scaling. Fig. 15 shows strong scaling results when training AF3 with a fixed global batch size of 32 while increasing the number of GPUs from 2 to 32. MegaFold consistently outperforms the baseline across all configurations, achieving a speedup of $1.44\times$ on 2 GPUs and $1.34\times$ on 32 GPUs, with an average improvement of $1.38\times$. The speedups marginally decrease with more GPUs due to increased communication overhead, which indicates the MegaFold’s gains are preserved as training becomes more distributed.

Weak scaling. Fig. 16 presents weak scaling results, where the batch size scales proportionally with the number of GPUs. MegaFold maintains consistent speedups across all configurations, delivering an average speedup of $1.37\times$ over PyTorch baseline from 2 to 32 GPUs. These results demonstrate that MegaFold’s sharding and communication strategy scales effectively with increasing problem size.

Together, the strong and weak scaling results show that MegaFold sustains its performance benefits as GPU count increases. This confirms that EvoSP-3D enables scalable training over quadratic pairwise representations without introducing prohibitive communication overheads.

D. Analysis

1) *Model Utilization and Scaling Behavior*: Fig. 17 reports model FLOPs utilization (MFU) when training AF3-style models at increasing parameter scales. MegaFold consistently

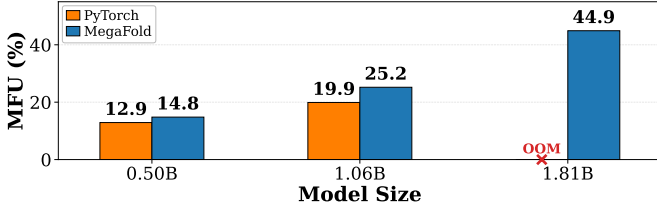


Fig. 17: Model FLOPs utilization (MFU) during AF3 training across increasing model sizes. MegaFold consistently improves MFU over the PyTorch baseline and remains trainable at larger model scales, while the baseline runs into OOM.

achieves higher MFU than PyTorch baseline across all evaluated configurations. At smaller model size (0.5B), utilization is limited by fixed overheads such as kernel launches and data movement, though MegaFold already provides a modest improvement. As model size increases to 1.06B, MegaFold sustains higher utilization (25.2% vs. 19.9%), indicating more effective use of GPU compute. At 1.81B, the PyTorch baseline runs into OOMs, while MegaFold continues to scale and reaches 44.9% MFU. This shows that MegaFold’s utilization gains are enabled by reduced activation memory pressure and kernel fragmentation, allowing training to proceed at scales that are otherwise infeasible. Overall, MegaFold shifts AF training toward a more compute-efficient regime as model size increases, while remaining bounded by memory and communication constraints inherent to 3D attention.

2) *Operation-Level Breakdown*: We analyze MegaFold’s impact at the operator level by breaking down GPU memory consumption and execution time across major operator groups in AF3, as shown in Fig. 18. We group operators into EvoAttention, transition layers, and remaining operators to isolate the effects of MegaFold’s kernel-level optimizations. MegaFold significantly reduces the memory footprint of the dominant operators in AF3. As shown in Fig. 18 (left), the largest reduction occurs in EvoAttention (4.24 $\times$), achieved by avoiding materialization of cubic attention maps in HBM through EvoFlash-3D. In addition, MegaFold reduces the memory consumption of transition layers (1.39 $\times$) and other operators (1.05 $\times$) by eliminating redundant intermediate activations via EvoFusion. These results confirm that MegaFold’s kernel level optimizations directly target the primary sources of activation memory pressure identified in § III-A. MegaFold also substantially reduces the execution time for the dominant operators in AF3. As shown in Fig. 18 (right), EvoAttention observes the largest relative speedup (2.5 $\times$), which is from reduction of global memory traffic and improved data locality from the EvoFlash-3D kernel. Transition layers are accelerated by 1.3 $\times$ due to EvoFusion, which reduces kernel launch overhead and immediate memory access through operator fusion. The execution time of remaining operators is also reduced (1.46 $\times$), which contributes to overall performance gains. Together, these results demonstrate that MegaFold’s kernel level optimizations effectively improve execution efficiency at the operator granularity.

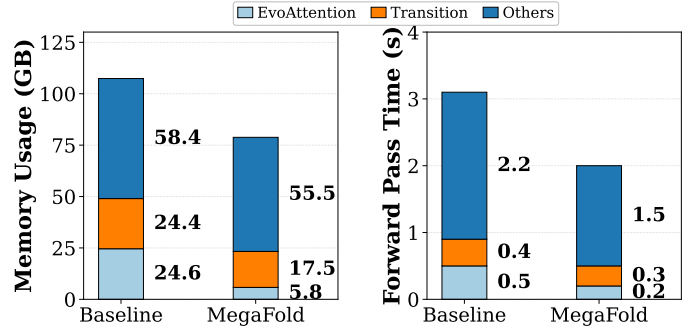


Fig. 18: Operation-level breakdown of peak activation memory (left) and forward execution time (right) for AF3 training. MegaFold reduces memory consumption and execution time across EvoAttention, transition layers, and other operators.

3) *Component Contribution*: Fig. 19 shows the contribution of individual optimizations on an NVIDIA H200. EvoPipe does not increase memory consumption, as its cache is generated offline, while EvoFlash-3D and EvoFusion both contribute significantly to extending the maximum trainable sequence length. In terms of execution time, MegaFold reduces per-iteration time by 1.44 $\times$ over PyTorch baseline through a combination of kernel-level and system-level optimizations, including EvoFlash-3D, EvoFusion, and EvoPipe. Each component targets a different source of inefficiency in AF3 training, and together they improve performance across the entire training pipeline.

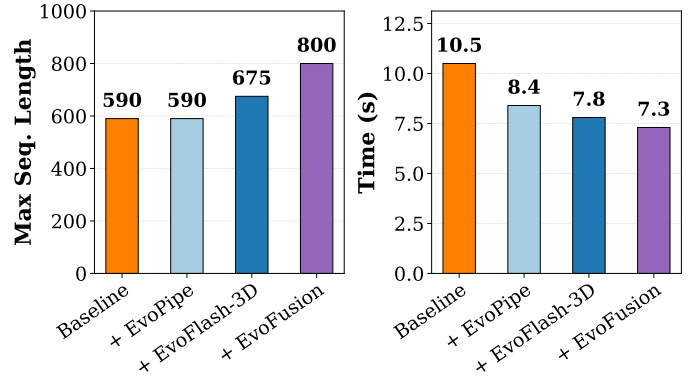


Fig. 19: Impact of MegaFold’s optimizations on AF3 training. Left: maximum trainable sequence length before OOM. Right: per-iteration training time.

4) *Scaling on different hardware*: Fig. 20 shows the maximum trainable sequence length on various NVIDIA H200-141GB and AMD MI325X-256GB GPU counts. MegaFold outperforms the PyTorch baseline in all tested configurations, enabling up to a 2.45 $\times$ & 2.72 $\times$ increase in input sequence lengths on NVIDIA and AMD hardware, respectively. Through a targeted set of optimizations that are designed to be cross-platform, with custom kernels written in Triton, MegaFold achieves consistent gains across multiple hardware vendors.

VI. RELATED WORK

Automated fusion and compilation systems. Mirage [24], TensorRT [25], OpenVINO [26], AITemplate [27], TVM [11],

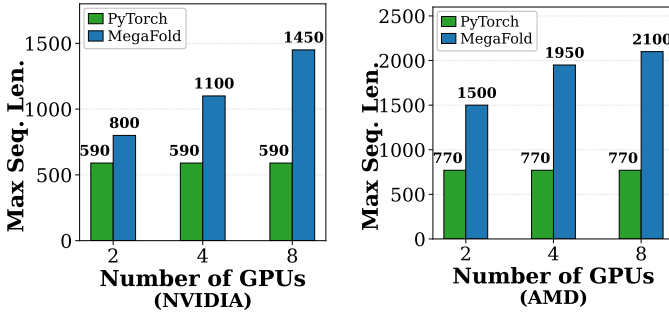


Fig. 20: AF3 trainability across various hardware vendors: NVIDIA H200-141GB (left) and AMD MI325x-256GB (right) with a fixed batch size. As GPU count increases, MegaFold scales to training on longer sequence lengths compared to the PyTorch baseline.

and IPEX [28] explore automated fusion and code-generation for deep neural networks. However, these systems primarily target inference workloads and do not support the backward pass required for training. NVFuser [29] provides automatic kernel fusion for training, but operates only on custom CUDA kernels and assumes relatively regular operator structures.

Custom kernel libraries. Liger [30], TransformerEngine [31], and xFormers [32] provide highly optimized primitives for transformer models. However, these libraries do not support the novel mixed-dimensional operators introduced in AF3, such as 3D EvoAttention. Composable-kernel (CK) [33] supports training optimizations on AMD GPUs but requires rewriting complex operators in HIP C++ and does not provide out-of-the-box support for AF3-style operators.

AlphaFold-specific system optimizations. Prior work has proposed system optimizations tailored to AlphaFold models. FastFold [9] introduces Dynamic Axial Parallelism (DAP) and kernel optimizations for the Evoformer module in AlphaFold2 [1]. ScaleFold [34] further improves AF2 training with data loading optimizations and specialized kernels for layer normalization and attention. However, these systems target AlphaFold2 and are not directly applicable to AF3, which introduces substantial architectural changes, including structured diffusion, and 3D attention over 2D pairwise representations. In addition, some AF-specific systems are not open-sourced or are limited to NVIDIA hardware.

VII. CONCLUSION

AlphaFold3-style models introduce 3D attention and mixed-dimensional operators that fundamentally differ from transformer models, making existing LLM-based optimizations ineffective. MegaFold addresses these challenges through cross-stack co-design, combining memory-efficient 3D attention kernels, communication-efficient sharding for pairwise representations, fused operator restructuring, and determinism-aware host-device pipeline. Our evaluation shows that MegaFold enables longer trainable sequences and faster training across both NVIDIA and AMD GPUs. These results demonstrate the

need for system design tailored to emerging scientific models beyond conventional transformer-centric optimizations.

ACKNOWLEDGMENTS

We sincerely appreciate the anonymous reviewers. Their insightful feedback helps significantly improve the quality of the paper. This research was supported by the National Science Foundation (NSF) under Grant No. 2441601 and 2308699. The work utilized the Delta and DeltaAI system at the National Center for Supercomputing Applications (NCSA) and Jetstream2 at Indiana University through allocation CIS240055 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296. The Delta advanced computing resource is a collaborative effort between the University of Illinois Urbana-Champaign and NCSA, supported by the NSF (award OAC 2005572) and the State of Illinois. This research also used resources of the National Energy Research Scientific Computing Center, a DOE Office of Science User Facility supported by the Office of Science of the U.S. Department of Energy (DOE) under Contract No. DE-AC02-05CH11231 using AI4Sci@NERSC award NERSC DDR-ERCAP0036206 awarded to AM. UIUC SSAIL Lab is supported by research funding and gifts from IBM, Google, Amazon, and AMD, including the Google ML and Systems Junior Faculty Award.

REFERENCES

- [1] J. Jumper, R. Evans, A. Pritzel *et al.*, “Highly accurate protein structure prediction with alphafold,” *Nature*, vol. 596, no. 7873, pp. 583–589, 2021.
- [2] J. Abramson, J. Adler, J. Dunger, and *et al.*, “Accurate structure prediction of biomolecular interactions with alphafold 3,” *Nature*, vol. 630, 2024.
- [3] G. Ahdriz, N. Bouatta, C. Floristean, S. Kadyan, Q. Xia, W. Gerecke, T. J. O’Donnell, D. Berenberg, I. Fisk, N. Zanichelli, B. Zhang, A. Nowaczynski, B. Wang, M. M. Stepniewska-Dziubinska, S. Zhang, A. Ojewole, M. Efe Guney, S. Biderman, A. M. Watkins, S. Ra, P. Ribalta Lorenzo, L. Nivon, B. Weitzner, Y.-E. Andrew Ban, S. Chen, M. Zhang, C. Li, S. L. Song, Y. He, P. K. Sorger, E. Mostaque, Z. Zhang, B. Richard, and M. AlQuraishi, “Openfold: Retraining alphafold2 yields new insights into its learning mechanisms and capacity for generalization,” *Nature Methods*, 2024.
- [4] R. Krishna, J. Wang, W. Ahern, P. Sturmfels, P. Venkatesh, I. Kalvet, G. R. Lee, F. S. Morey-Burrows, I. Anishchenko, I. R. Humphreys, R. McHugh, D. Vafeados, X. Li, G. A. Sutherland, A. Hitchcock, C. N. Hunter, A. Kang, E. Brackenbrough, A. K. Bera, M. Baek, F. DiMaio, and D. Baker, “Generalized biomolecular modeling and design with rosettafold all-atom,” *Science*, vol. 384, no. 6693, 2024.
- [5] Z. Lin, H. Akin, R. Rao, B. Hie, Z. Zhu, W. Lu, N. Smetanin, R. Verkuil, O. Kabeli, Y. Shmueli, A. dos Santos Costa, M. Fazel-Zarandi, T. Sercu, S. Candido, and A. Rives, “Evolutionary-scale prediction of atomic level protein structure with a language model,” *bioRxiv*, 2022.
- [6] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, M. Gallé, and *et al.*, “BLOOM: A 176B-parameter Open-Access Multilingual Language Model,” 2022, bigScience Workshop.
- [7] S. A. Jacobs, M. Tanaka, C. Zhang, M. Zhang, R. Y. Aminadabi, S. L. Song, S. Rajbhandari, and Y. He, “System optimizations for enabling training of extreme long sequence transformer models,” in *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing*, ser. PODC ’24. New York, NY, USA: Association for Computing Machinery, 2024.
- [8] V. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoenybi, and B. Catanzaro, “Reducing activation recomputation in large transformer models,” 2022.

- [9] S. Cheng, X. Zhao, G. Lu, J. Fang, Z. Yu, T. Zheng, R. Wu, X. Zhang, J. Peng, and Y. You, "Fastfold: Reducing alphafold training time from 11 days to 67 hours," 2023.
- [10] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "Flashinfer: Efficient and customizable attention engine for llm inference serving," *arXiv preprint arXiv:2501.01005*.
- [11] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Tvm: An automated end-to-end optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, 2018.
- [12] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating High-Performance tensor programs for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 863–879. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/zheng>
- [13] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. K. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, S. Zhang, M. Suo, P. Tillet, X. Zhao, E. Wang, K. Zhou, R. Zou, X. Wang, A. Mathews, W. Wen, G. Chanan, P. Wu, and S. Chintala, "Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024.
- [14] H. Liu, M. Zaharia, and P. Abbeel, "Ring attention with blockwise transformers for near-infinite context," 2023.
- [15] P. Wang, "alphafold3-pytorch," <https://github.com/lucidrains/alphafold3-pytorch>, 2025.
- [16] N. Shazeer, "Glu variants improve transformer," 2020. [Online]. Available: <https://arxiv.org/abs/2002.05202>
- [17] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [18] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," 2018.
- [19] P. Tillet, H. T. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.
- [20] P. D. Bank, "Protein data bank," *Nature New Biol*, vol. 233, no. 223, 1971.
- [21] S. L. Song, B. Kruff, M. Zhang *et al.*, "DeepSpeed4science initiative: Enabling large-scale scientific discovery through sophisticated ai system technologies," 2023. [Online]. Available: <https://arxiv.org/abs/2310.04610>
- [22] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," 2016.
- [23] S. Li, "Getting started with distributed data parallel," PyTorch Tutorials, 2025, accessed: Apr. 9, 2026. [Online]. Available: https://docs.pytorch.org/tutorials/intermediate/ddp_tutorial.html
- [24] M. Wu, X. Cheng, S. Liu, C. Shi, J. Ji, K. Ao, P. Velliengiri, X. Miao, O. Padon, and Z. Jia, "Mirage: A multi-level superoptimizer for tensor programs," in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. Boston, MA: USENIX Association, Jul. 2025.
- [25] NVIDIA, "TensorRT: SDK for high-performance deep learning inference on NVIDIA GPUs." 2019. [Online]. Available: <https://github.com/NVIDIA/TensorRT>
- [26] Intel, "OpenVINO: Open-source software toolkit for optimizing and deploying deep learning models." 2019. [Online]. Available: <https://github.com/openvinotoolkit/openvino>
- [27] Meta, "AI Template: Python framework which renders neural network into high performance CUDA/HIP C++ code." 2022. [Online]. Available: <https://github.com/facebookincubator/AITemplate>
- [28] Intel, "ipex-llm: Intel® LLM Library for PyTorch," 2017. [Online]. Available: <https://github.com/intel/ipex-llm>
- [29] NVIDIA, "NVFuser: A Fusion Code Generator for NVIDIA GPUs," 12 2021. [Online]. Available: <https://github.com/NVIDIA/Fuser>
- [30] P.-L. Hsu, Y. Dai, V. Kothapalli, Q. Song, S. Tang, S. Zhu, S. Shimizu, S. Sahni, H. Ning, and Y. Chen, "Liger kernel: Efficient triton kernels for llm training," 2025.
- [31] NVIDIA, "TransformerEngine: A library for accelerating Transformer models on NVIDIA GPUs, including using 8-bit floating point (FP8) precision on Hopper, Ada and Blackwell GPUs, to provide better performance with lower memory utilization in both training and inference." 2023. [Online]. Available: <https://github.com/NVIDIA/TransformerEngine>
- [32] Meta, "xformers: Hackable and optimized Transformers building blocks, supporting a composable construction." 2021. [Online]. Available: <https://github.com/facebookresearch/xformers>
- [33] C. Liu, J. Zhang, L. Qin, Q. Zhang, L. Huang, S. Wang, A. Chang, C. Lai, I. Silin, A. Osewski, P. Chen, R. Geyyer, H. Chen, T. Shah, X. Zhou, and J. Yan, "Composable Kernel," 2023. [Online]. Available: https://github.com/ROCm/composable_kernel
- [34] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," 2020.
- [35] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.
- [36] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "Flashattention-3: Fast and accurate attention with asynchrony and low-precision," 2024.
- [37] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," 2020.
- [38] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on gpu clusters using megatron-lm," 2021.
- [39] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," 2023.
- [40] B. Zhang and R. Sennrich, "Root mean square layer normalization," 2019.
- [41] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '20. New York, NY, USA: Association for Computing Machinery, 2020.
- [42] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Mathews, and S. Li, "Pytorch fsdp: Experiences on scaling fully sharded data parallel," 2023.
- [43] L. Atkinson, "trifast," <https://github.com/latkins/trifast>, 2025.
- [44] A. Golden, S. Hsia, F. Sun, B. Acun, B. Hosmer, Y. Lee, Z. DeVito, J. Johnson, G.-Y. Wei, D. Brooks, and C.-J. Wu, "Is flash attention stable?" 2024.
- [45] O. Dekel, "Blockbuster, part 1: Block-level ai operator fusion," 2025.
- [46] Y. Shi, S. Zheng, G. Ke, Y. Shen, J. You, J. He, S. Luo, C. Liu, D. He, and T.-Y. Liu, "Benchmarking graphormer on large-scale molecular modeling datasets," 2023.
- [47] J. Ross, B. Belgodere, V. Chenthamarakshan *et al.*, "Large-scale chemical language representations capture molecular structure and properties," *Nature Machine Intelligence*, vol. 4, 2022.
- [48] R. Irwin, S. Dimitriadis, J. He, and E. J. Bjerrum, "Chemformer: a pre-trained transformer for computational chemistry," *Machine Learning: Science and Technology*, vol. 3, no. 1, 2022.
- [49] F. Zhu, A. Nowaczynski, R. Li, J. Xin, Y. Song, M. Marcinkiewicz, S. B. Eryilmaz, J. Yang, and M. Andersch, "Scalefold: Reducing alphafold initial training time to 10 hours," 2024.
- [50] M. Zvyagin, A. Brace, K. Hippe, Y. Deng, B. Zhang, C. O. Bohorquez, A. Clyde, B. Kale, D. Perez-Rivera, H. Ma, C. M. Mann, M. Irvin, J. G. Pauloski, L. Ward, V. Hayot, M. Emani, S. Foreman, Z. Xie, D. Lin, M. Shukla, W. Nie, J. Romero, C. Dallago, A. Vahdat, C. Xiao, T. Gibbs, I. Foster, J. J. Davis, M. E. Papka, T. Brettin, R. Stevens, A. Anandkumar, V. Vishwanath, and A. Ramanathan, "Genslms: Genome-scale language models reveal sars-cov-2 evolutionary dynamics," *bioRxiv*, 2022.
- [51] T. Nguyen, J. Brandstetter, A. Kapoor, J. K. Gupta, and A. Grover, "Climax: A foundation model for weather and climate," in *Proceedings of Machine Learning Research*, vol. 202, 2023.
- [52] T. P. Consortium, "Training of 1-Trillion Parameter Scientific AI Begins," <https://www.hpcwire.com/2023/11/13/training-of-1-trillion-parameter-scientific-ai-begins/>, 2023.

- [53] J. Abramson, J. Adler, J. Dunger, R. Evans, T. Green, A. Pritzel, O. Ronneberger, L. Willmore, A. J. Ballard, J. Bambrick, S. W. Bodenstein, D. A. Evans, C.-C. Hung, M. O’Neill, D. Reiman, K. Tunyasuvunakool, Z. Wu, A. Žemgulytė, E. Arvaniti, C. Beattie, O. Bertolli, A. Bridgland, A. Cherepanov, M. Congreve, A. I. Cowen-Rivers, A. Cowie, M. Figurnov, F. B. Fuchs, H. Gladman, R. Jain, Y. A. Khan, C. M. R. Low, K. Perlin, A. Potapenko, P. Savy, S. Singh, A. Stecula, A. Thillaisundaram, C. Tong, S. Yakneen, E. D. Zhong, M. Zielinski, A. Židek, V. Bapst, P. Kohli, M. Jaderberg, D. Hassabis, and J. M. Jumper, “Accurate structure prediction of biomolecular interactions with AlphaFold 3,” *Nature*, 2024.
- [54] RDKit, “RdKit: Open-source cheminformatics software,” <https://www.rdkit.org/>, 2013.
- [55] E. Simon and J. Silberg, “The Illustrated AlphaFold,” <https://elanapearl.github.io/blog/2024/the-illustrated-alphafold/>, 2023.
- [56] A. Grattafiori, A. Dubey, A. Jauhri *et al.*, “The llama 3 herd of models,” 2024.
- [57] M. H. Daniel Han and U. team, “Unsloth,” 2023. [Online]. Available: <http://github.com/unslothai/unsloth>
- [58] Jan 2025. [Online]. Available: <https://catalog.ngc.nvidia.com/orgs/nvidia/teams/dgxc-benchmarking/resources/llama31-8b-dgxc-benchmarking-a>
- [59] SambaNova Systems, “Sambaflow compiler overview,” <https://docs.sambanova.ai/developer/latest/compiler-overview.html>, 2024, accessed: August 20, 2025.
- [60] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [61] PyTorch, “torch.utils.data,” 2025, accessed: 2025-12-01. [Online]. Available: <https://docs.pytorch.org/docs/stable/data.html>
- [62] G. Leclerc, A. Ilyas, L. Engstrom, S. M. Park, H. Salman, and A. Madry, “Ffcv: Accelerating training by removing data bottlenecks,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.12517>
- [63] D. G. Murray, J. Simsa, A. Klimovic, and I. Indyk, “tf.data: A machine learning data processing framework,” 2021. [Online]. Available: <https://arxiv.org/abs/2101.12127>

APPENDIX

A. Triton

Triton is an embedded domain-specific-language in python that allows users to program GPUs with code-generation support for diverse hardware (such as NVIDIA and AMD GPUs), removing the need to rewrite kernels in separate vendor specific DSLs such as CUDA and ROCm. Triton operates on the thread-block level, allowing users to specify how work should decompose across thread-blocks. The Triton compiler then makes a decision on how work is decomposed within the threads constituent to a block. All our code-listings are written in this perspective, detailing the decomposition of work across thread-blocks.

B. Computations of EvoAttention & EvoFlash-3D Backwards Pass

To complement § IV-A, we illustrate the computations involved in the forward and backward pass of EvoAttention in table I as well as a code-listing for the backwards pass of EvoFlash-3D in 2, detailing its computations next.

The backwards pass operates in two stages, in the first stage, the dV & dK tensors are computed (lines 31-47). We schedule each thread-block to compute a chunk of rows in each tensor by fixing a chunk of keys & values and iterating through the queries. First, each thread-block zero-initializes its local answers of dV & dK (line 33) and retrieves a fixed chunk of keys and values (line 34-35). Then, the inner-loop (lines 36-46) iterates over the queries and: (1) extracts the correct logsumexp in m and $D=dO^T O$ (line 38). The tensor

```

1 # Omit indexing on batch/head/hid. dim for brevity
2 def fused_attn_bwd(dO,Q,K,V,bias,l,m,D,O):
3     D = preprocess(dO,O)
4     dk, dv = dv_dk(dO,Q,K,V,bias,l,m,D)
5     dBias = zeros(bias.shape)
6     dq = dq_dbias(dO,Q,K,V,bias,l,m,D,dBias)
7     return dq, dk, dv, dbias
8 # Stage 1, compute: (1)dv, (2)dP, (3)dS, (4)dk
9 def dv_dk(dO,Q,K,V,bias,l,m,D):
10    j = tl.program_id(0)
11    dk = dv = zeros((bx, d))
12    kj, vj = k[j*bx:(j+1)*bx, :], \
13            v[j*bx:(j+1)*bx, :]
14    for i in range(N/by):
15        qiT = q[i*by:(i+1)*by, :].T
16        mi, di = m[i*by:(i+1)*by], D[i*by:(i+1)*by]
17        dOi = dO[i*by:(i+1)*by, :]
18        sT = tl.matmul(kj, qiT) + \
19              bias[j*bx:(j+1)*bx,i*by:(i+1)*by]
20        pT = tl.exp(sT-mi)
21        dv += tl.matmul(pT, dOi)
22        dpT = tl.matmul(v, dOi.T)
23        dST = pT * (dpT - di)
24        dk += tl.matmul(dST, qiT.T)
25    return dk, dv
26 # Stage 2, compute: (1)dS, (2)dbias, (3)dq
27 def dq_dbias(dO,Q,K,V,bias,l,m,D,dBias):
28    i = tl.program_id(0)
29    dq = zeros((by, d))
30    qi = Q[i*by:(i+1)*by, :]
31    doi = dO[i*by:(i+1)*by, :]
32    mi, di = m[i*by:(i+1)*by], D[i*by:(i+1)*by]
33    for j in range(N/bx):
34        kjT, vjT = K[j*bx:(j+1)*bx, :].T, \
35                V[j*bx:(j+1)*bx, :].T
36        s = tl.matmul(qi, kjT) + \
37              bias[i*by:(i+1)*by,j*bx:(j+1)*bx]
38        p = tl.exp(s-mi)
39        dp = tl.matmul(doi, vjT)
40        ds = p * (dp-di)
41        tl.atomic_add(dBias, ds)
42        dq += tl.matmul(ds, kjT.T)
43    return dq

```

Listing 2: EvoFlash-3D backwards pass.

D is precomputed prior to launching the backward pass. (2) computes the transpose of the attention-logits, pT (line 40-42). (3) Multiplies the transposed attention-logits with the incoming gradients to compute dV (line 43). (4) Computes dp -transposed by multiplying the values and the incoming gradients (line 44). (5) Computes dS -transposed (line 45). (6) Finally, computes dK by multiplying dS -transposed by the queries q (line 46).

In the second stage, the dQ and $dBias$ tensors are computed (lines 49-65). In contrast with the first stage, we schedule a thread-block to compute a chunk of rows in each tensor by fixing a chunk of queries and iterating through the values and keys. The preamble prior to the loop is similar to stage one. The inner-loop (lines 55-64) first computes a chunk of the s , p , dp , and ds tensors. Then, the $dBias$ gradients are computed by reducing the local ds chunk through an `atomic_add` to the corresponding $dBias$ memory location; doing so mitigates the need to hold partially reduced values in scratchpad memories and avoids impacting the occupancy of our kernel. Finally, the dQ -chunk is computed by multiplying ds by the keys, k .

C. Computations of LayerNorm & Linear Layer

To complement § IV-C, we illustrate the computations involved in the forward and backward pass of a layer-norm & linear-layer in table II. The c_1 and c_2 variables are intermediate values that are used to compute the gradients of the input tokens.

TABLE I: EvoAttention’s forward and backward passes.

Forward Pass	Backward Pass (given $dO = \frac{\partial \mathcal{L}}{\partial O}$)
	$dV = P^T dO$
	$dP = dOV^T$
$S = QK^T$	$dS = P \odot (dP$
$+ \text{Bias}_{\text{bcast}}$	$\left. - \text{rowsum}(dO^T O) \right)_D$
$P = \text{Softmax}(S)$	$dK = Q^T dS$
$O = PV$	$dBias = \sum_{\text{bcast}} dS$
	$dQ = dS K$

We additionally detail the forward and backward passes in listings 3 and 4 respectively. Since the forward pass is detailed in § IV-C, we detail only its backwards pass next.

Our backward pass launches two kernels (lines 34-63 in listing 3). The first kernel computes the gradients of the linear-layer weights, the weights of the LayerNorm parameters, as well as extra c_1 , c_2 , and w_{dy} data, intermediate variables required to compute the gradients of the token_p tensor (the mathematical formulae for these intermediate values are described in table II). The second kernel then consumes these data to finally compute the gradients of the token_p tensor.

The first kernel first reads the input token tensor and re-normalizes the data by applying the LayerNorm weights and bias (lines 36-40), and then zero-initializes a $[b_m, b_k]$ -sized tile where a local tile of dO_1 will be computed. An inner-loop (lines 42-48) then traverses across the linear-layer’s hidden dimension and computes: (1) a tile of dW_1 , atomically aggregating this to main-memory, where the overall result of dW_1 is stored; (2) updates the local tile of dO_1 . After the inner-loop terminates, the local tile of dO_1 is fully computed and is used to compute w_{dy} (line 49), which is then used to compute c_1 & c_2 by atomically aggregating local values of c_1 & c_2 to main-memory, where their overall results are stored (lines 50-51). Finally, the LayerNorm weights dw and db are computed (lines 52-53) and the w_{dy} tensor is returned to be consumed by the second kernel in the backwards pass. In the second kernel (lines 56-63), each thread-block computes a chunk of rows of $d\text{token}_p$ by first reading the original input tokens from the token tensor, normalizing the loaded input and finally using the c_1 , c_2 , w_{dy} & normalized inputs to compute $d\text{token}_p$.

D. EvoPipe

To complement § IV-D, we illustrate the default and optimized training loop prior/post application of EvoPipe’s determinism-aware host-device pipeline in listing 5.

E. Activation Checkpointing Setup

Given the large activations memory produced during AF3 training, activation-checkpointing (AC) must be enabled otherwise it is not possible to train AF3 on moderate sequence lengths. AC is systematically applied to every sub-layer within

```

1 def transition_fwd(token, eps, w, b, W1, W2):
2     mean, rstd = mean_var(token, eps)
3     o1 = fused_ln_linear_fwd(token, eps, mean, \
4                             rstd, w, b, W1)
5     o2 = fused_swiglu_fwd(o1)
6     return o = matmul(o2, W2)
7 def transition_bwd(dO, w, b, W1, W2):
8     ... = fused_swiglu_bwd(...)
9     do1 = ... # Omit computing earlier gradients
10    c1 = c2 = zero_init((M,1))
11    dW1, dw, db, c1, c2, wdy = \
12        dW1_dw_db_dc1_dc2(...)
13    dtokenp = dtokenp(...)
14    def mean_var(token, eps):
15        row, k = token[tl.program_id(0)], \
16            token.shape(-1)
17        mean = tl.sum(row, axis=-1) / k
18        var = tl.sum((row-mean)**2, axis=0) / k
19        rstd = 1/tl.sqrt(var+eps)
20        return mean, rstd
21    def fused_ln_linear_fwd(token, eps, mean, rstd, w,
22                            b, W1):
23        m, n = swizzle(tl.program_id(0))
24        o2 = zeros((bm, bn))
25        for k in range(K/bk): # K is the hid. dimension
26            # Load (bm, bk)-sized chunk
27            x = token[m*bm:(m+1)*bm, k*bk:(k+1)*bk]
28            w1, b1 = w[k*bk:(k+1)*bk], b[k*bk:(k+1)*bk]
29            # Load (bk, bn)-sized chunk
30            y = W1[k*bk:(k+1)*bk, n*bn:(n+1)*bn]
31            xhat = (x - mean) * rstd * w1 + b1
32            o2 += tl.matmul(xhat, y) # (bm,bk) x (bk,bn)
33    return o2

```

Listing 3: Fused Transition layer forward pass.

```

1 def transition_bwd(dO, w, b, W1, W2):
2     ... = fused_swiglu_bwd(...)
3     do1 = ... # Omit computing earlier gradients
4     c1 = c2 = zero_init((M,1))
5     dW1, dw, db, c1, c2, wdy = \
6         dW1_dw_db_dc1_dc2(...)
7     dtokenp = dtokenp(...)
8     # computes dw, db, c1 and c2
9     def dW1_dw_db_dc1_dc2(dO2, dW1, token, mean, rstd,
10                            w, b, W1, c1_ptr, c2_ptr):
11        m, k = swizzle(tl.program_id(0))
12        x = token[m*bm:(m+1)*bm, k*bk:(k+1)*bk]
13        mean1, rstd1 = mean[m*bm:(m+1)*bm], \
14            rstd[m*bm:(m+1)*bm]
15        xhat = (x - mean1) * rstd1
16        yiT = tl.trans(xhat*w + b) # Layer-norm output
17        dO1 = zero_init((bm, bk))
18        for n in range(N/bn): # N is the ffn hid. dim
19            dO2 = dO2[m*bm:(m+1)*bm, n*bn:(n+1)*bn]
20            dW1 = matmul(yiT, dO2)
21            tl.atomic_add(dW1[k*bk:(k+1)*bk, \
22                            n*bn:(n+1)*bn], dW1)
23            W1T = W1[k*bk:(k+1)*bk, n*bn:(n+1)*bn].T
24            dO1 += matmul(dO2, W1T)
25        wdy = dO1 * w
26        tl.atomic_add(c1_ptr, sum(xhat * wdy, dim=1)/K)
27        tl.atomic_add(c2_ptr, sum(wdy, dim=1)/K)
28        tl.atomic_add(dw_ptr, sum(dO1 * xhat, dim=0))
29        tl.atomic_add(db_ptr, sum(dO1, dim=0))
30        return wdy
31    # computes dtokenp
32    def dtokenp(c1, c2, wdy, token, mean, rstd):
33        i = tl.program_id(0)
34        xi = token[i] # Block operates on a row.
35        cli, c2i, meani, rstdi, wdyi = c1[i], c2[i], \
36            mean[i], rstd[i], wdy[i]
37        xhati = (xi-meani) * rstdi
38        dtokenpi = (wdyi - (cli * xhati + c2i)) * rstdi
39        return dtokenpi

```

Listing 4: Fused Transition layer backward pass.

each layer in AF3 modules. Using the terminology in the original AF3 paper, we apply AC to the following:

- MSA module: AC is applied on each OuterProductMean,

TABLE II: EvoFusion for Common AlphaFold Operators

Forward Pass	Backward Pass (given $dO = \frac{\partial \mathcal{L}}{\partial O_2}$)
$O_1 = \underbrace{\frac{\text{token}_p - \mathbb{E}[\text{token}_p]}{\sqrt{\text{Var}(\text{token}_p) + \epsilon}}}_{\text{token}_p} \odot w + b$ $O_2 = O_1 W_1$	$dW_1 = O_1^T dO_2$ $dO_1 = dO_2 W_1^T$ $dw = dO_1 \odot \text{token}_p$ $db = \sum_{\text{dimension}=0} dO_1$ $d\text{token}_p = \frac{1}{\text{Var}(\text{token}_p)} \left(dO_1 \odot w - c_1 \odot \text{token}_p - c_2 \right)$ $c_1 = \frac{1}{N_{\text{token}}} \sum_{\text{dimension}=1} \text{token}_p \odot (dO_1 \odot w)$ $c_2 = \frac{1}{N_{\text{token}}} \sum_{\text{dimension}=1} dO_1 \odot w$

```

1 def generate_cache(data):
2     cache = {}
3     for inp in data:
4         cache[inp] = (atom_features(inp), \
5                        msa_features(inp))
6     return cache
7 def train_loop(data, af, opt, loss_fn):
8     """Main training loop.
9     data: input data-source.
10    af: the af-model to train.
11    """
12    for inp, out in data:
13        # Extract atom feature info
14        atom_feat = atom_features(inp)
15        # Extract msa feature info
16        msa_feat = msa_features(inp)
17        p_atom_feat, p_msa_feat = preprocess(atom_feat,
18                                              msa_feat)
19        tokenize_repr = tokenize(p_atom_feat,
20                                p_msa_feat)
21        output = loss_fn(af(tokenize_repr), output)
22        output.backward()
23        opt.step()
24 def cached_train(data, af, opt, loss_fn, cache):
25     """Main training loop.
26     data: input data-source.
27     af: the af-model to train.
28     """
29     for inp, out in data:
30         atom_feat, msa_feat = cache[inp]
31         p_atom_feat, p_msa_feat = preprocess(atom_feat,
32                                              msa_feat)
33         tokenize_repr = tokenize(p_atom_feat,
34                                 p_msa_feat)
35         output = loss_fn(af(tokenize_repr), output)
36         output.backward()
37         opt.step()
38 # Stage 1: Populate cache.
39 cache = generate_cache(data)
40 # Stage 2: train.
41 cached_train(data, af, opt, loss, cache)

```

Listing 5: Main training loop after application of EvoPipe.

MSAPairWeightedAveraging, Transition, and Pairwise-Block sub-layer

- Template module: AC is applied on each PairwiseBlock sub-layer
- Pairformer module: AC is applied on each PairwiseBlock, AttentionPairBias, and Transition sub-layer
- Diffusion module: AC is applied on each AttentionPairBias, Transition sub-layer

When AC is applied, all intermediate activations within

the sub-layer are discarded during the forward pass, and only the input tensors are retained. In the backward pass, since the intermediate activations have been discarded, an additional forward pass is performed using the retained inputs to recompute necessary activations for gradient calculations. AC accordingly makes a trade-off between speed and memory, sacrificing speed for reduced peak memory consumption. On the other hand, our optimizations achieve the best of both worlds: faster training speeds with reduced peak memory consumption.

F. Fraction of theoretical peak achieved of Individual Optimizations

We compare the fraction of theoretical peak achieved when applying MegaFold EvoFlash-3D and EvoFusion optimizations to AF3 operators against PyTorch in Fig. 21. MegaFold EvoFlash-3D and EvoFused operators achieve upto 4 and 6% of peak throughput, respectively. Comparatively, PyTorch achieves <1% and 3% of peak throughput, indicating our optimizations effectively reduce the number of reads/writes to HBM through fusion to accelerate performance. Note that the input sizes to these kernels are relatively small (typical of AF3 models), resulting in a significantly lower achievable throughput compared to larger models like LLMs.

G. Implementation Validation

Fused kernels often modify reductions and may change the numerical stability of operators. Therefore, to verify the correctness of our optimizations, we train MegaFold & a PyTorch implementation of AF3 for 120 iterations with a sequence length of 384 using the Protein Data Bank (the defacto dataset to train AF3), plotting the training loss at each step in Fig. 22. MegaFold matches the per-step training loss of the baseline, indicating that our optimizations do not sufficiently impact AF3’s convergence.

H. Comparison against Hand-written EvoAttention Kernels

We compare the runtime performance of MegaFold’s memory-efficient EvoAttention operator against TriFast [43],

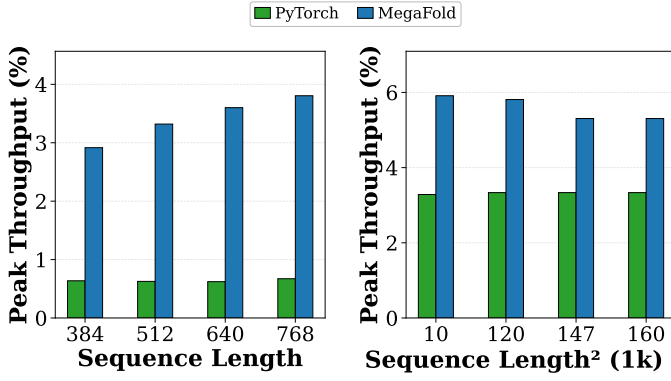


Fig. 21: Comparing fraction of theoretical throughput achieved of MegaFold optimizations and the PyTorch baseline. Left and Right are the EvoAttention and Transition operators respectively. MegaFold optimizations and custom kernels consistently outperform PyTorch.

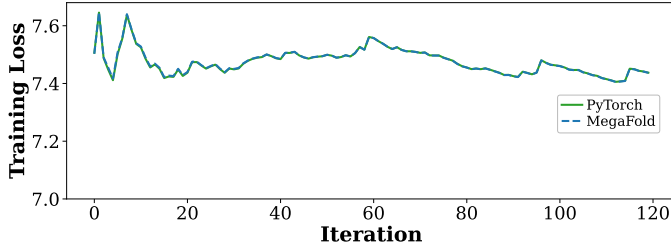


Fig. 22: Comparing training loss between MegaFold and PyTorch of AF3 across 120 steps. MegaFold and PyTorch exhibit the same convergence.

a hand-written EvoAttention implementation heavily used by industry in Fig. 23. Across all benchmarked sequence lengths, MegaFold’s EvoAttention operator - EvoFlash-3D - is consistently $1.2\times$ - $2\times$ faster than TriFast. This speedup is attributed to the backwards pass. TriFast launches three kernels to compute gradients: (1) dK & dV , (2) dQ and (3) $dBias$. On the other hand, MegaFold launches two kernels: (1) dK & dV , and (2) dQ & $dBias$, comparatively reducing kernel launch overhead.

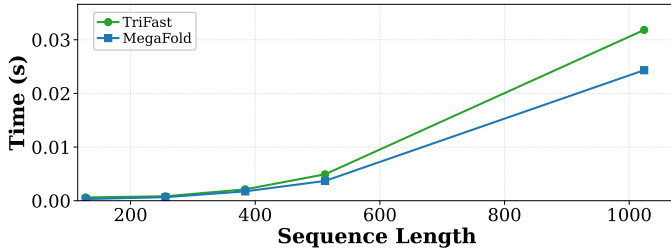


Fig. 23: Comparing the runtime performance of TriFast and MegaFold’s EvoAttention operators across different sequence lengths. Timing aggregates forward and backwards pass. MegaFold consistently outperforms TriFast across all sequence lengths.